

RNA-seq Processing Workflow

Technical Documentation

Raw reads → quality control → gene-level count matrix

Snakemake · FastQC · fastp · Salmon · pytximport · MultiQC

Contents

1. Overview and Scope
2. Repository Structure
3. Environment and Dependencies
4. Configuration System
5. Pipeline Steps in Detail
6. Design Decisions and Rationale
7. Known Limitations and Assumptions
8. Running the Workflow
9. Validation Approach

1. Overview and Scope

This workflow turns raw RNA-seq FASTQ reads into a quality-controlled, gene-level count matrix suitable for downstream analysis (e.g. differential expression, which is explicitly out of scope for this pipeline itself). Summary quality-control information is produced alongside the count matrix.

Design constraints that shaped every decision below:

- The workflow must be reproducible — identical inputs always produce identical outputs, without hidden state.
- It must support typical RNA-seq input data, including both paired-end and single-end libraries.
- It must produce gene-level counts suitable for downstream statistical analysis.
- Quality-control information must be generated and summarized in one place.
- It must remain simple and runnable on a local machine — no cluster or cloud dependency.

The pipeline is orchestrated with **Snakemake**, a workflow manager that expresses each processing step as a rule with declared inputs and outputs. Snakemake builds a dependency graph (DAG) from these rules, runs only what is missing or stale, and parallelizes independent steps automatically.

```
raw FASTQ → FastQC → fastp (trim) → Salmon (index + quant) → tx2gene → gene_counts  
→ MultiQC
```

2. Repository Structure

Path	Purpose
<code>Snakefile</code>	All pipeline rules (the workflow definition itself).
<code>config.test.yaml</code>	Configuration for the pipeline's own built-in test fixture — samples, reference paths, output location.
<code>environment.yml</code>	Conda environment definition listing every tool the pipeline needs, with pinned versions.
<code>test-data/</code>	A small, checked-in FASTQ + reference fixture used to exercise the pipeline end-to-end without any external download. Not a general-purpose reference — see Section 4.
<code>results/<name>/</code>	Generated output, namespaced per config (e.g. <code>results/test/</code>). Not checked into version control — fully regenerable.
<code>logs/</code>	Per-rule, per-sample log files from every tool invocation. Not checked into version control.

Running the pipeline against a real dataset does not require modifying any of the files above: it only requires a new config file (copied from `config.test.yaml`) pointing at different samples, a different reference, and a different `results_dir`. The pipeline logic itself has no dataset-specific assumptions baked in.

3. Environment and Dependencies

All tools are installed into a single conda environment defined by `environment.yml`, activated once before running Snakemake. A single shared environment (rather than a separate one per rule) was chosen deliberately to keep the first version of this pipeline simple to set up and reason about.

Tool	Version	Role
snakemake-minimal	9.26	Workflow orchestration
fastqc	0.12	Raw-read quality control
fastp	1.3	Adapter/quality trimming
salmon	2.7	Transcriptome indexing and pseudo-alignment / quantification
multiqc	1.35	Aggregated QC reporting
pandas	3.0	Dependency of pytximport
pytximport	0.13.0 (pip)	Transcript-to-gene summarization of Salmon output

```
conda env create -f environment.yml
conda activate rnaseq-workflow
```

4. Configuration System

Every run is driven by a YAML config file with three top-level keys:

```
results_dir: results/test          # where this run's outputs go

reference:
  transcriptome_fasta: <path>      # transcript-level FASTA (not a genome)
  gtf: <path>                     # annotation, used to derive gene mapping

samples:
  <sample_name>:
    r1: <path>                    # required
    r2: <path>                    # optional: omit for single-end samples
```

4.1 Paired-end vs. single-end samples

A sample is treated as paired-end if it has an `r2` key, and single-end otherwise. The Snakefile computes two sample lists at parse time (`PE_SAMPLES` , `SE_SAMPLES`) and routes each sample to the matching rule variant (e.g. `fastp_pe` vs. `fastp_se`) using Snakemake's `wildcard_constraints` , so a single config can mix both library types freely.

4.2 Output isolation via `results_dir`

Every rule's output path is prefixed with the config's `results_dir` value rather than a hardcoded `results/` . This means two different configs (e.g. the test fixture and a real dataset) never share or collide on output files, and a QC aggregation step (MultiQC) run against one config's directory tree can never accidentally pick up another run's leftover files.

4.3 Why there is no default config

Snakemake *merges* a config file passed via `--configfile` with any default set via a `configfile:` directive inside the Snakefile — it does not replace it. Nested dictionaries (like `samples`) are merged key-by-key, so a default config and a `--configfile` override would silently combine into a union of both configs' sample sets. To avoid this failure mode entirely, the Snakefile declares no default `configfile:` at all. A config must always be passed explicitly:

```
snakemake --configfile config.test.yaml --cores 4
```

Omitting `--configfile` fails immediately with a clear error (`KeyError: 'results_dir'`) rather than silently running with an incomplete or wrongly-merged configuration.

5. Pipeline Steps in Detail

RULE fastqc_raw

Purpose: Quality control on the raw, untrimmed reads — per-base quality scores, GC content, adapter content, duplication levels, and overrepresented sequences.

Input: the sample's raw FASTQ file(s) (one for single-end, two for paired-end) as given in the config.

Output: a per-sample directory of FastQC HTML/ZIP reports.

```
fastqc --outdir <output_dir> <r1> [<r2>]
```

A single rule handles both library types: its input function returns one or two files depending on whether the sample has an `r2`, and FastQC accepts a variable number of input files natively.

RULE fastp_pe / fastp_se

Purpose: Adapter and quality trimming. fastp also produces its own JSON/HTML QC report, which MultiQC later ingests as a second, independent QC data source alongside FastQC.

Input: the sample's raw FASTQ file(s).

Output: trimmed FASTQ file(s), plus a JSON and HTML report.

```
fastp -i <r1> -I <r2> -o <trimmed_r1> -O <trimmed_r2> -j <report.json> -h  
<report.html>    # paired-end  
fastp -i <r1>      -o <trimmed_r1>                -j <report.json> -h  
<report.html>    # single-end
```

RULE salmon_index

Purpose: Build a Salmon index from the transcriptome FASTA. This is a one-time step per reference, shared by all samples.

Input: the transcriptome FASTA named in `config["reference"]["transcriptome_fasta"]`.

Output: a Salmon index directory.

```
salmon index -t <transcriptome.fasta> -i <index_dir> -p <threads>
```

This builds a plain (non-decoy-aware) transcriptome index. Decoy-aware indexing improves mapping specificity by also including the genome as a decoy, but requires the genome sequence in addition to the transcriptome; it was left out of this first version to keep the reference requirements to a single transcriptome FASTA + GTF. See Section 7.

RULE salmon_quant_pe / salmon_quant_se

Purpose: Pseudo-align each sample's trimmed reads against the transcriptome index and estimate per-transcript abundance. Salmon is used instead of a genome aligner (e.g. STAR/HISAT2) plus a separate read-counting tool (e.g. featureCounts): it maps reads directly against transcripts, which is faster and sufficient when only a gene-level count matrix is needed (no alignment positions or splice-junction information required).

Input: the sample's trimmed FASTQ file(s) and the Salmon index.

Output: a `quant.sf` file with per-transcript length, effective length, TPM, and estimated read count.

```
salmon quant -i <index> -l A -1 <r1> -2 <r2> -p <threads> -o <outdir> #  
paired-end  
salmon quant -i <index> -l A -r <r1> -p <threads> -o <outdir> #  
single-end
```

`-l A` auto-detects the sequencing library's strandedness (e.g. unstranded, forward-stranded, reverse-stranded) from the reads themselves rather than assuming one — important because getting strandedness wrong silently misassigns reads on genes with overlapping antisense transcripts.

RULE tx2gene

Purpose: Derive a transcript-to-gene mapping table from the annotation, needed to collapse Salmon's transcript-level estimates to gene level.

Input: the GTF named in `config["reference"]["gtf"]`.

Output: a two-column `transcript_id` / `gene_id` table.

```
pytximport create-map -i <genes.gtf> -o <tx2gene.tsv>
```

Assumption: this step assumes a standard GTF with `transcript_id` / `gene_id` attributes (Ensembl-style). A GFF3 file (`ID` / `Parent`-style attributes) or a nonstandard GTF would need the mapping tool's field-name options adjusted; see the corresponding comment in the Snakefile.

RULE gene_counts

Purpose: Combine every sample's `quant.sf` using the transcript-to-gene mapping to produce the final gene-level count matrix — the pipeline's primary deliverable.

Input: all samples' `quant.sf` files and the transcript-to-gene mapping.

Output: a CSV count matrix, genes as rows, one column per sample.

```
pytximport run -i <sample1/quant.sf> -i <sample2/quant.sf> ... \  
-t salmon -m <tx2gene.tsv> -o <gene_counts.csv> -of csv
```

Counts are non-integer floating point values — standard for EM-based quantification, where reads compatible with multiple transcripts are fractionally assigned. Most downstream tools (e.g. DESeq2) accept this natively.

RULE multiqc

Purpose: Aggregate every tool's QC output (FastQC, fastp, Salmon) into a single browsable HTML report, satisfying the requirement that QC information be summarized alongside the results.

Input: all samples' FastQC directories, fastp reports, and Salmon `quant.sf` files (scoped to the current run's `results_dir` only — see Section 4.2).

Output: a single `multiqc_report.html`.

```
multiqc <results_dir>/qc <results_dir>/salmon --outdir <results_dir>/qc \  
--filename multiqc_report.html
```

RULE all

Purpose: The default target. Depends on the gene count matrix and the MultiQC report, so a plain `snakemake --configfile ... --cores N` invocation builds everything.

6. Design Decisions and Rationale

Decision	Rationale
Snakemake over a plain shell script or Nextflow	Dependency-aware and incremental (re-runs only what changed), scales naturally to more samples, without introducing Nextflow's steeper DSL for a first version.
Salmon (pseudo-alignment) over STAR/HISAT2 + featureCounts	No genome index, no BAM sorting/indexing step; faster; sufficient when only gene-level counts (not alignment positions) are needed. Also avoids STAR's small-genome index-size tuning gotcha entirely.
pytximport for gene-level summarization	Standard, well-tested reimplementation of R's <code>tximport</code> with a scriptable command-line interface, avoiding a dependency on R.
Single shared conda environment	Keeps setup to one command; per-rule environments (Snakemake supports this) were deferred as an unnecessary complexity for a first version.
<code>results_dir</code> as a config key	Prevents different runs (e.g. the test fixture vs. a real dataset) from overwriting or contaminating each other's outputs — discovered as a real issue when a QC aggregation step picked up a previous run's stale files.
No default <code>configfile:</code>	Snakemake merges rather than replaces configs; a default would silently combine two configs' sample sets when a different one is passed via <code>--configfile</code> . See Section 4.3.

7. Known Limitations and Assumptions

- **GTF attribute naming:** the transcript-to-gene mapping step assumes Ensembl-style `transcript_id` / `gene_id` attributes. A GFF3 or nonstandard GTF may require extra options.
- **No decoy-aware Salmon index:** a plain transcriptome index is used. Decoy-aware indexing (adding the genome as a decoy) improves mapping specificity but requires the genome sequence as an additional reference input.
- **No automatic reference/read consistency check:** nothing in the pipeline verifies that the supplied reads, transcriptome, and GTF actually correspond to the same organism/assembly. A mismatch would run "successfully" but silently produce a very low mapping rate.
- **Sample names are used directly as path components** (e.g. `results/<dir>/salmon/<sample>/`) with no sanitization; a sample name containing `/` or spaces would break path construction.
- **Differential expression analysis is explicitly out of scope** for this pipeline — its output is a count matrix intended as input to a separate downstream analysis.

8. Running the Workflow

8.1 One-time setup

```
conda env create -f environment.yml
conda activate rnaseq-workflow
```

8.2 Dry run (recommended before a real run)

```
snakemake -n --configfile config.test.yaml --cores 4
```

Prints the planned jobs and their dependency order without executing anything — useful for confirming all inputs resolve correctly.

8.3 Full run

```
snakemake --configfile config.test.yaml --cores 4
```

8.4 Running against a different dataset

Copy `config.test.yaml` to a new file, point `reference` and `samples` at the new data, and give it its own `results_dir`. No changes to the `Snakefile` are needed.

8.5 Outputs

Within a config's `results_dir`:

- `counts/gene_counts.csv` — the gene-level count matrix.
- `qc/multiqc_report.html` — the aggregated QC report.

9. Validation Approach

Beyond a successful run, the pipeline's output was validated with a series of consistency and sanity checks during development:

- **Internal consistency:** the sum of the gene-level count matrix matches Salmon's own reported number of mapped reads, to within a small residual explained by a handful of transcripts present in the transcriptome FASTA but absent from the GTF's transcript set.
- **Idempotency:** re-running the pipeline with no input changes correctly reports nothing to do, confirming that every rule's inputs/outputs are declared correctly.
- **Plausibility of QC metrics:** mapping rates, trimming statistics, and FastQC pass/warn/fail flags were checked for values within normal ranges for real Illumina RNA-seq data (not just the tiny synthetic test fixture).
- **Biological sanity check:** on a real, published RNA-seq dataset with a known experimental perturbation, the gene targeted by that perturbation showed a count change in the expected direction and approximate magnitude — evidence that the full chain (trimming, pseudo-alignment, gene-level summarization) produces biologically sensible results, not merely internally-consistent numbers.